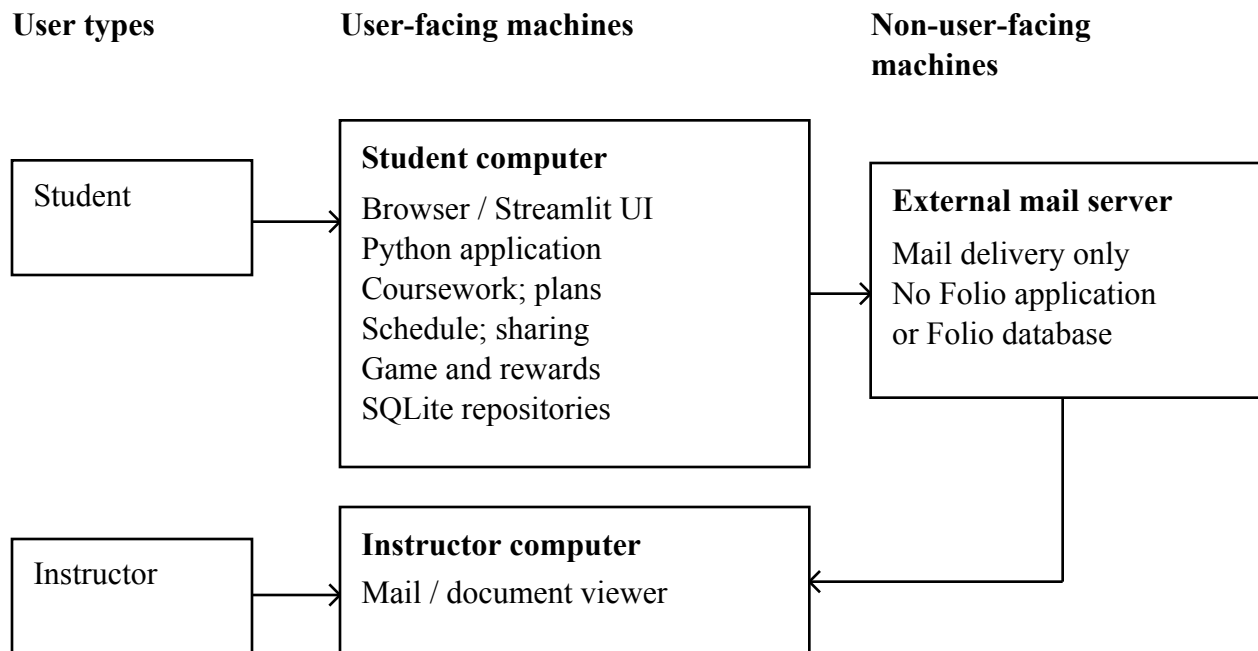


Folio Software Design

Caleb Brooks and Nicholas Dunzelman | September 23, 2026

1 Architecture

Folio is a local, single-user Python application with a Streamlit browser interface and SQLite storage. The diagram shows the planned deployment. The instructor uses shared documents or email; instructor participation is optional.



The browser sends student actions to application services. Services apply planning, scheduling, sharing, and game rules, then use repositories to save records. Domain rules do not depend on Streamlit or document-generation libraries. The local application works without an external database. Only optional email delivery uses an external mail service.

2 Module functionality and interfaces

The following interfaces describe the intended application. Existing demos exercise selected rules; these interfaces do not imply that the integrated application is implemented. Each service returns its result or validation errors to the UI. Identifiers retain associations among coursework, plans, and tasks.

Coursework

Manual entry and CSV import use the same field validation. Course schedules, assessments, deadlines, and scores provide academic context independently of rewards. Imported records retain their source and original row numbers for error reporting.

ManualEntry(data); EditOrDeleteCoursework(id, action, changes): validate changes and return the saved record or field errors. Deletion checks dependent plans and tasks before changing records.

ParseCSV(file); ValidateAndDeduplicate(rows): check required columns, validate each row, and store valid nonduplicate records. Return accepted/rejected counts and row errors; an unreadable file or missing required columns saves no records.

Plans and feedback

The student chooses coursework and a template, edits its suggested tasks, and saves a plan. Suggestions are entered from optional instructor feedback. Accepting or adapting a suggestion prepares a revision; the saved plan changes only when the student chooses to save that revision. Rejecting a suggestion leaves the plan unchanged.

select_coursework(coursework_id); choose_template(template_id): return the selected records and suggested tasks, or a selection error when either identifier is absent.

edit_task(task_id, changes): validate the proposed task fields and update the plan draft; return the changed task or field errors.

review_suggestion(suggestion_id, decision, changes=None): record accept, reject, or adapt and return the resulting draft. Adapt uses the student's changes; an unknown decision is rejected.

save_plan(plan_id): save the chosen coursework, template, tasks, and student-approved revision together, then return the saved plan. No instructor response is required.

Schedule and completion

Schedule combines tasks from active plans while preserving each task's identity. An overlap exists when $A.start < B.end$ and $A.end > B.start$. Adjacent intervals do not conflict. The UI identifies every conflict and leaves rescheduling choices to the student.

get_tasks(); get_tasks_for_plan(plan_id): return all tasks or one plan's tasks. The UI filters these records into daily and weekly views and displays each task's course or plan.

add_task(task); find_conflicts(task): validate the task and report overlapping tasks. Identical tasks within one plan are rejected; distinct tasks from different plans remain separate.

reschedule_task(task_id, new_start_time, new_end_time): validate the proposed interval. If conflicts exist, return them and keep the saved interval; otherwise save the new interval. A task cannot end at or before its start.

remove_task(task_id): remove the task from its plan and the combined schedule, preserving other tasks.

configure_reminder(task_id, settings): validate and save the task's reminder configuration; the interface reports invalid settings. The reminder service displays due reminders while Folio is running.

set_task_complete(task_id, completed); get_plan_progress(plan_id): save completion, return progress, and produce congratulations on a change to complete. Progress is completed tasks divided by all tasks in that plan, or zero for an empty plan. Reward eligibility is evaluated separately.

Sharing and export

Sharing receives only student-selected academic fields. A single allowlist filter builds the data used by every output. Private progress, reward, game, and preference records never enter that output structure.

`select_share_content(plan_id, selected_fields); get_shareable_academic_data(plan_id, selected_fields)`: return the selected academic values. Unknown or private fields are excluded even if requested.

`export_pdf(academic_data); export_docx(academic_data)`: generate the selected plan with `fpdf2` or `python-docx` and return the file or an export error. The M1 PDF demo uses `Playwright`; `fpdf2` is the selected implementation library.

`prepare_instructor_email(academic_data)`: construct a subject and body using Python email tools and return an unsent preview. Sending requires a separate student action after review; editing or cancelling the preview does not send it.

Game and rewards

The optional game is one-player 3×3 tic-tac-toe. Game state and reward state remain separate from academic records. Completing a task earns one move at most once per task identifier. Without an earned move, the student loses the turn and the computer moves. XP and points do not replace or consume earned moves.

`RewardState.complete_task(task_id, task_weight=1)`: ignore processed task identifiers; otherwise add $10 \times \text{weight}$ XP, $5 \times \text{weight}$ points, and one earned move. Reject a weight that is not a positive integer; save the processed identifier with the award.

`consume_game_move(); lose_turn(); get_reward_state()`: consume one available move, leave rewards unchanged for a lost turn, or return reward totals. Level, badge, and cosmetic rules use the separate reward state; they never alter plans or coursework.

`TicTacToeGame.new_game(); make_move(position)`: randomly select the starter on a cleared board or apply a legal move. Occupied/out-of-range cells and finished games reject a move. A controller checks earned moves before accepting a human move.

`computer_move(); check_winner()`: select a legal computer move using minimax with alpha-beta pruning, or return win/loss/tie/unfinished. The controller records each finished result once and starts a new game. Disabling game/rewards leaves planning available.

Persistence

`StoreCoursework(action, record); StorePlanData(plan, tasks, feedback)`: save related academic changes in a SQLite transaction using stable primary and foreign keys. Failed operations roll back; success returns after commit.

`StoreRemindersAndPreferences(reminders, preferences); StoreGameAndRewardState(game, rewards)`: save settings and private state separately. Persist processed task identifiers, remaining moves, and game totals to prevent duplicate awards after restart.

`RestoreSavedApplicationState()`: restore the most recently saved coursework, plans, tasks, reminders, preferences, feedback, and game/reward state with their relationships. Export files remain local files, separate from the database.

3 GUI sketches

Coursework

Course [CSE 4101] Work [Milestone 1] Type [Assignment v]

Due date [2026-09-28] Score [80] Course schedule [Edit]

Date format: YYYY-MM-DD. Score: numeric.

[Save] [Edit selected] [Delete selected]

CSV type [Course schedule / Assessment v]

Required columns and formats [View guide]

[Choose CSV] [Import]

Imported: 2 Rejected: 1

Row 3: due_date must use YYYY-MM-DD.

Manual fields use visible validation rules. The CSV guide shows required columns and formats for the chosen record type before import; results identify rejected rows and fields. The selected coursework becomes the context for the Plan screen.

Plan

Coursework [Milestone 1 v] Template [Project v]

Suggested task [Draft requirements] [Edit task]

Schedule [2026-09-24 14:00 to 15:00] [Save plan]

Instructor suggestion [Review the acceptance criteria]

[Accept] [Reject] [Adapt]

Adapted task [Review export privacy criteria]

[Save revision] [Keep current plan]

Accept and Adapt prepare a revision for the student to review. Keep current plan discards that proposed revision. Planning and saving remain available when no instructor suggestion exists.

Schedule

[Daily] [Weekly] Plan progress: 1 of 2 complete (50%)

CSE 4101 / M1 Draft requirements 14:00–15:00

[Complete] [Reschedule] [Reminder settings]

Conflict: Review test plan, 14:30–15:30. Choose another time.

Task complete. Congratulations!



Daily and weekly views show tasks from active plans. A conflict message identifies the other task; the student chooses the change. Completion updates that plan's progress bar.

Share plan

Academic fields: [x] Course [x] Coursework [x] Planned tasks

Preview: CSE 4101 / Milestone 1 / Draft requirements

[Export PDF] [Export DOCX] [Prepare email]

Email preview: [To] [Subject] [Body]

[Edit] [Send reviewed email] [Cancel]

The preview and every export use the same filtered academic data. Private progress, rewards, and game information are absent. Preparing an email leaves it unsent.

Optional game and rewards

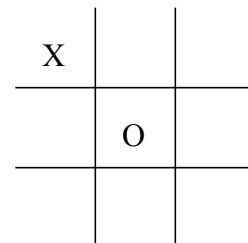
Game/rewards [On / Off] Earned moves: 1

Your turn: choose an empty square.

Wins: 0 Losses: 0 Ties: 0

XP: 10 Points: 5

Level [view] Badges [view] Cosmetics [view]



The board accepts a human move only when an earned move is available. The game displays the current turn and totals. Turning the feature off leaves the academic screens usable.